

Greedy Graph Algorithms

Dijkstra's Algorithm, Kruskal's MST, Prim's MST, Boruvka's MST

Dr. Sriramya

Name : Jaasiel J S

Graph Algorithm Practice Questions

Dijkstra's • Kruskal's • Prim's • Boruvka's

20 Questions | 5 per Algorithm | Easy → Hard

Dijkstra's Algorithm — 5 Questions

Q1. Find the shortest path from vertex 0 to all other vertices.

Easy *Single-source shortest path*

Input:

```
n = 5, source = 0 edges =  
[(0,1,4), (0,2,1), (2,1,2), (1,3,1), (2,3,5), (3,4,3)]
```

Expected Output:

```
Distances from 0: [0, 3, 1, 4, 7] Path to 1: 0 → 2 → 1 Path to 3: 0 → 2 →  
1 → 3 Path to 4: 0 → 2 → 1 → 3 → 4
```

main.py	Output
<pre> 1 import heapq 2 3 def dijkstra(n, edges, source, undirected=False): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((v,w)) 7 if undirected: 8 g[v].append((u,w)) 9 dist=[float("inf")]*n 10 parent=[-1]*n 11 dist[source]=0 12 pq=[(0,source)] 13 while pq: 14 d,u=heapq.heappop(pq) 15 if d!=dist[u]: 16 continue 17 for v,w in g[u]: 18 nd=d+w 19 if nd<dist[v]: 20 dist[v]=nd 21 parent[v]=u 22 heapq.heappush(pq,(nd,v)) 23 return dist,parent 24 25 def path_to(parent, v): 26 path=[] 27 while v!=-1: 28 path.append(v); v=parent[v] 29 return path[::-1] 30 31 n=5; source=0 32 edges=[(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)] 33 dist,parent=dijkstra(n,edges,source) 34 print("Distances from 0:", dist) 35 for v in [1,3,4]: 36 print(f"Path to {v}:", " -> ".join(map(str,path_to(parent,v)))) </pre>	<pre> Distances from 0: [0, 3, 1, 4, 7] Path to 1: 0 -> 2 -> 1 Path to 3: 0 -> 2 -> 1 -> 3 Path to 4: 0 -> 2 -> 1 -> 3 -> 4 </pre>

Q2. Find the shortest path from source 0 to destination 4 in a weighted directed graph.

Medium *Single-pair shortest path*

Input:

```
n = 5, source = 0, dest = 4 edges =
[(0,1,10),(0,2,3),(1,3,2),(2,1,4),(2,3,8),(2,4,2),(3,4,5)]
```

Expected Output:

```
Shortest distance: 5 Path: 0 → 2 → 4
```

main.py	Output
<pre> 1 import heapq 2 3 def dijkstra(n, edges, source, undirected=False): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((v,w)) 7 if undirected: 8 g[v].append((u,w)) 9 dist=[float("inf")]*n 10 parent=[-1]*n 11 dist[source]=0 12 pq=[(0,source)] 13 while pq: 14 d,u=heapq.heappop(pq) 15 if d!=dist[u]: 16 continue 17 for v,w in g[u]: 18 nd=d+w 19 if nd<dist[v]: 20 dist[v]=nd 21 parent[v]=u 22 heapq.heappush(pq,(nd,v)) 23 return dist,parent 24 25 def path_to(parent, v): 26 path=[] 27 while v!=-1: 28 path.append(v); v=parent[v] 29 return path[::-1] 30 31 n=5; source=0; dest=4 32 edges=[(0,1,10),(0,2,3),(1,3,2),(2,1,4),(2,3,8),(2,4,2),(3,4,5)] 33 dist,parent=dijkstra(n,edges,source) 34 print("Shortest distance:", dist[dest]) 35 print("Path:", " -> ".join(map(str,path_to(parent,dest)))) </pre>	<pre> Shortest distance: 5 Path: 0 -> 2 -> 4 </pre>

Q3. A graph has multiple paths to the same destination. Find the minimum cost path from node 0 to node 5.

Medium *Dijkstra with path reconstruction*

Input:

```
n = 6, source = 0 edges = [(0,1,7),(0,2,9),(0,5,14),(1,2,10),(1,3,15),
(2,3,11),(2,5,2),(3,4,6),(4,5,9)]
```

Expected Output:

```
Shortest distance from 0 to 5: 11 Path: 0 → 2 → 5
```

main.py	Output
<pre> 1 import heapq 2 3 def dijkstra(n, edges, source, undirected=False): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((v,w)) 7 if undirected: 8 g[v].append((u,w)) 9 dist=[float("inf")]*n 10 parent=[-1]*n 11 dist[source]=0 12 pq=[(0,source)] 13 while pq: 14 d,u=heapq.heappop(pq) 15 if d!=dist[u]: 16 continue 17 for v,w in g[u]: 18 nd=d+w 19 if nd<dist[v]: 20 dist[v]=nd 21 parent[v]=u 22 heapq.heappush(pq,(nd,v)) 23 return dist,parent 24 25 def path_to(parent, v): 26 path=[] 27 while v!=-1: 28 path.append(v); v=parent[v] 29 return path[::-1] 30 31 n=6; source=0; dest=5 32 edges=[(0,1,7),(0,2,9),(0,5,14),(1,2,10),(1,3,15),(2,3,11),(2,5,2),(3,4,6),(4,5,9)] 33 dist,parent=dijkstra(n,edges,source) 34 print("Shortest distance from 0 to 5:", dist[dest]) 35 print("Path:", " -> ".join(map(str,path_to(parent,dest)))) </pre>	<pre> Shortest distance from 0 to 5: 11 Path: 0 -> 2 -> 5 </pre>

Q4. Determine if node 3 is reachable from node 0. If yes, output the shortest distance and path.

Medium *Reachability + shortest path*

Input:

```
n = 5, source = 0 edges = [(0,1,5),(1,2,3),(2,4,1),(3,4,2),(0,4,10)]
```

Expected Output:

```
Node 3 is reachable. Shortest distance from 0 to 3: 11 Path: 0 → 1 → 2 → 4 → 3
```

main.py	Output
<pre> 1 import heapq 2 3 def dijkstra(n, edges, source, undirected=False): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((v,w)) 7 if undirected: 8 g[v].append((u,w)) 9 dist=[float("inf")]*n 10 parent=[-1]*n 11 dist[source]=0 12 pq=[(0,source)] 13 while pq: 14 d,u=heapq.heappop(pq) 15 if d!=dist[u]: 16 continue 17 for v,w in g[u]: 18 nd=d+w 19 if nd<dist[v]: 20 dist[v]=nd 21 parent[v]=u 22 heapq.heappush(pq,(nd,v)) 23 return dist,parent 24 25 def path_to(parent, v): 26 path=[] 27 while v!=-1: 28 path.append(v); v=parent[v] 29 return path[::-1] 30 31 n=5; source=0; dest=3 32 edges=[(0,1,5),(1,2,3),(2,4,1),(3,4,2),(0,4,10)] 33 dist,parent=dijkstra(n,edges,source,undirected=True) 34 if dist[dest] != float("inf"): 35 print("Node 3 is reachable.") 36 print("Shortest distance from 0 to 3:", dist[dest]) 37 print("Path:", " -> ".join(map(str,path_to(parent,dest)))) 38 else: 39 print("Node 3 is not reachable.") </pre>	<pre> Node 3 is reachable. Shortest distance from 0 to 3: 11 Path: 0 -> 1 -> 2 -> 4 -> 3 </pre>

Q5. Find shortest distances from source 0. Identify the vertex that is hardest (longest shortest path) to reach.

Hard All-pairs analysis

Input:

```
n = 6, source = 0 edges =
[(0,1,2), (0,2,4), (1,2,1), (1,3,7), (2,4,3), (3,5,1), (4,3,2), (4,5,5)]
```

Expected Output:

```
Distances: [0, 2, 3, 8, 6, 9] Hardest vertex to reach: 5 (distance = 9)
Path to 5: 0 → 1 → 2 → 4 → 3 → 5
```

main.py	Output
<pre> 1 import heapq 2 3 def dijkstra(n, edges, source, undirected=False): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((v,w)) 7 if undirected: 8 g[v].append((u,w)) 9 dist=[float("inf")]*n 10 parent=[-1]*n 11 dist[source]=0 12 pq=[(0,source)] 13 while pq: 14 d,u=heapq.heappop(pq) 15 if d!=dist[u]: 16 continue 17 for v,w in g[u]: 18 nd=d+w 19 if nd<dist[v]: 20 dist[v]=nd 21 parent[v]=u 22 heapq.heappush(pq,(nd,v)) 23 return dist,parent 24 25 def path_to(parent, v): 26 path=[] 27 while v!=-1: 28 path.append(v); v=parent[v] 29 return path[::-1] 30 31 n=6; source=0 32 edges=[(0,1,2),(0,2,4),(1,2,1),(1,3,7),(2,4,3),(3,5,1),(4,3,2),(4,5,5)] 33 dist,parent=dijkstra(n,edges,source) 34 finite=[(d,v) for v,d in enumerate(dist) if d < float("inf")] 35 hard_dist,hard=max(finite) 36 print("Distances:", dist) 37 print(f"Hardest vertex to reach: {hard} (distance = {hard_dist})") 38 print("Path to 5:", " -> ".join(map(str,path_to(parent,5)))) </pre>	<pre> Distances: [0, 2, 3, 8, 6, 9] Hardest vertex to reach: 5 (distance = 9) Path to 5: 0 -> 1 -> 2 -> 4 -> 3 -> 5 </pre>

Kruskal's Algorithm — 5 Questions

Q1. Find the MST and its total weight using Kruskal's Algorithm.

Easy *Basic MST*

Input:

```
n = 4, m = 5 edges = [(0,1,10), (0,2,6), (0,3,5), (1,3,15), (2,3,4)]
```

Expected Output:

```
Edges in MST: [(2,3,4), (0,3,5), (0,1,10)] Total weight of MST: 19
```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): 3 self.p=list(range(n)); self.r=[0]*n 4 def find(self,x): 5 while self.p[x]!=x: 6 self.p[x]=self.p[self.p[x]]; x=self.p[x] 7 return x 8 def union(self,a,b): 9 a=self.find(a); b=self.find(b) 10 if a==b: return False 11 if self.r[a]<self.r[b]: a,b=b,a 12 self.p[b]=a 13 if self.r[a]==self.r[b]: self.r[a]+=1 14 return True 15 16 def kruskal(n,edges): 17 d=DSU(n); mst=[] 18 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 19 if u!=v and d.union(u,v): mst.append((u,v,w)) 20 return mst,sum(w for _,_,w in mst) 21 22 n=4 23 edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)] 24 mst,total=kruskal(n,edges) 25 print("Edges in MST:", mst) 26 print("Total weight of MST:", total) </pre>	<pre> Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)] Total weight of MST: 19 </pre>

Q2. Apply Kruskal's Algorithm. If multiple edges have the same weight, choose the one with the smaller source vertex.

Easy *MST with tie-breaking*

Input:

```

n = 5, m = 7 edges =
[(0,1,2), (0,3,6), (1,2,3), (1,3,8), (1,4,5), (2,4,7), (3,4,9)]

```

Expected Output:

```

Edges in MST: [(0,1,2), (1,2,3), (1,4,5), (0,3,6)] Total weight of MST: 16

```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): 3 self.p=list(range(n)); self.r=[0]*n 4 def find(self,x): 5 while self.p[x]!=x: 6 self.p[x]=self.p[self.p[x]]; x=self.p[x] 7 return x 8 def union(self,a,b): 9 a=self.find(a); b=self.find(b) 10 if a==b: return False 11 if self.r[a]<self.r[b]: a,b=b,a 12 self.p[b]=a 13 if self.r[a]==self.r[b]: self.r[a]+=1 14 return True 15 16 def kruskal(n,edges): 17 d=DSU(n); mst=[] 18 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 19 if u!=v and d.union(u,v): mst.append((u,v,w)) 20 return mst,sum(w for _,_,w in mst) 21 22 n=5 23 edges=[(0,1,2),(0,3,6),(1,2,3),(1,3,8),(1,4,5),(2,4,7),(3,4,9)] 24 mst,total=kruskal(n,edges) 25 print("Edges in MST:", mst) 26 print("Total weight of MST:", total) </pre>	<pre> Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)] Total weight of MST: 16 </pre>

Q3. Determine the MST weight. Then add edge (1,5,1) and find the new MST weight.

Medium *MST update on edge addition*

Input:

```
n = 6, m = 8 edges =
[(0,1,4), (0,2,3), (1,2,1), (1,3,2), (2,3,4), (3,4,2), (4,5,6), (3,5,5)]
```

Expected Output:

```
Original MST weight: 13 Edges: [(1,2,1), (1,3,2), (3,4,2), (0,2,3), (3,5,5)]
After adding (1,5,1): New MST weight: 9 Edges:
[(1,2,1), (1,5,1), (1,3,2), (3,4,2), (0,2,3)]
```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): 3 self.p=list(range(n)); self.r=[0]*n 4 def find(self,x): 5 while self.p[x]!=x: 6 self.p[x]=self.p[self.p[x]]; x=self.p[x] 7 return x 8 def union(self,a,b): 9 a=self.find(a); b=self.find(b) 10 if a==b: return False 11 if self.r[a]<self.r[b]: a,b=b,a 12 self.p[b]=a 13 if self.r[a]==self.r[b]: self.r[a]+=1 14 return True 15 16 def kruskal(n,edges): 17 d=DSU(n); mst=[] 18 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 19 if u!=v and d.union(u,v): mst.append((u,v,w)) 20 return mst,sum(w for _,_,w in mst) 21 22 n=6 23 edges=[(0,1,4),(0,2,3),(1,2,1),(1,3,2),(2,3,4),(3,4,2),(4,5,6),(3,5,5)] 24 mst,total=kruskal(n,edges) 25 print("Original MST weight:", total) 26 print("Edges:", mst) 27 new_edges=edges+[(1,5,1)] 28 mst2,total2=kruskal(n,new_edges) 29 print("After adding (1,5,1):") 30 print("New MST weight:", total2) 31 print("Edges:", mst2)</pre>	<pre> Original MST weight: 13 Edges: [(1, 2, 1), (1, 3, 2), (3, 4, 2), (0, 2, 3), (3, 5, 5)] After adding (1,5,1): New MST weight: 9 Edges: [(1, 2, 1), (1, 5, 1), (1, 3, 2), (3, 4, 2), (0, 2, 3)]</pre>

Q4. The graph may be disconnected. Find all MSTs (one per connected component) and report total weight.

Medium *MST on disconnected graph (forest)*

Input:

```
n = 7, m = 7 edges =
[(0,1,3), (0,2,5), (1,2,2), (3,4,4), (3,5,6), (4,5,1), (6,6,0)]
```

Expected Output:

```
Component 1 MST edges: [(1,2,2), (0,1,3)] weight: 5 Component 2 MST edges:
[(4,5,1), (3,4,4)] weight: 5 Component 3: isolated vertex 6 Total MST
(spanning forest) weight: 10
```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): 3 self.p=list(range(n)); self.r=[0]*n 4 def find(self,x): 5 while self.p[x]!=x: 6 self.p[x]=self.p[self.p[x]]; x=self.p[x] 7 return x 8 def union(self,a,b): 9 a=self.find(a); b=self.find(b) 10 if a==b: return False 11 if self.r[a]<self.r[b]: a,b=b,a 12 self.p[b]=a 13 if self.r[a]==self.r[b]: self.r[a]+=1 14 return True 15 16 def kruskal(n,edges): 17 d=DSU(n); mst=[] 18 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 19 if u!=v and d.union(u,v): mst.append((u,v,w)) 20 return mst,sum(w for _,_,w in mst) 21 22 n=7 23 edges=[(0,1,3),(0,2,5),(1,2,2),(3,4,4),(3,5,6),(4,5,1),(6,6,0)] 24 mst,total=kruskal(n,edges) 25 # Group MST edges by connected component. 26 d=DSU(n) 27 for u,v,w in edges: 28 if u!=v: d.union(u,v) 29 comps={} 30 for v in range(n): comps.setdefault(d.find(v),[]).append(v) 31 for idx,(root,verts) in enumerate(sorted(comps.items(),key=lambda x:min(x[1]),1)): 32 ce=[e for e in mst if e[0] in verts and e[1] in verts] 33 if len(verts)==1: 34 print(f"Component {idx}: isolated vertex {verts[0]}") 35 else: 36 print(f"Component {idx} MST edges: {ce} weight: {sum(e[2] for e in ce)}") 37 print("Total MST (spanning forest) weight:", total) </pre>	<pre> Component 1 MST edges: [(1, 2, 2), (0, 1, 3)] weight: 5 Component 2 MST edges: [(4, 5, 1), (3, 4, 4)] weight: 5 Component 3: isolated vertex 6 Total MST (spanning forest) weight: 10 </pre>

Q5. Find the MST. Then identify which non-MST edge, if added, would replace the most expensive MST edge.

Hard *MST sensitivity analysis*

Input:

```

n = 5, m = 8 edges =
[(0,1,1), (0,2,5), (1,2,3), (1,3,4), (2,3,2), (2,4,6), (3,4,7), (0,4,9)]

```

Expected Output:

```

MST edges: [(0,1,1), (2,3,2), (1,2,3), (1,3,4)] weight: 10 Non-MST edges:
[(0,2,5), (2,4,6), (3,4,7), (0,4,9)] Best swap: add (2,4,6), remove (1,3,4)
New MST weight: 12

```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): 3 self.p=list(range(n)); self.r=[0]*n 4 def find(self,x): 5 while self.p[x]!=x: 6 self.p[x]=self.p[self.p[x]]; x=self.p[x] 7 return x 8 def union(self,a,b): 9 a=self.find(a); b=self.find(b) 10 if a==b: return False 11 if self.r[a]<self.r[b]: a,b=b,a 12 self.p[b]=a 13 if self.r[a]==self.r[b]: self.r[a]+=1 14 return True 15 16 def kruskal(n,edges): 17 d=DSU(n); mst=[] 18 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 19 if u!=v and d.union(u,v): mst.append((u,v,w)) 20 return mst,sum(w for _,_,w in mst) 21 22 from collections import defaultdict, deque 23 24 def max_edge_on_path(n,mst,s,t): 25 g=defaultdict(list) 26 for u,v,w in mst: 27 g[u].append((v,w)); g[v].append((u,w)) 28 q=deque([(s,-1,[])]) 29 seen={s} 30 while q: 31 u,_,path=q.popleft() 32 if u==t: 33 return max(path,key=lambda x:x[2]) if path else None 34 for v,w in g[u]: 35 if v not in seen: 36 seen.add(v); q.append((v,u,path+[(u,v,w)])) 37 38 n=5 39 edges=[(0,1,1),(0,2,5),(1,2,3),(1,3,4),(2,3,2),(2,4,6),(3,4,7),(0,4,9)] 40 mst,total=kruskal(n,edges) 41 non=[e for e in edges if e not in mst] 42 best=None 43 for e in non: 44 rem=max_edge_on_path(n,mst,e[0],e[1]) 45 new_total=total + e[2] - rem[2] 46 cand=(new_total,e,rem) 47 if best is None or cand < best: best=cand 48 print("MST edges:", mst) 49 print("MST weight:", total) 50 print("Non-MST edges:", non) 51 print("Best alternative swap: add", best[1], "remove", best[2]) 52 print("Resulting tree weight:", best[0]) </pre>	<pre> MST edges: [(0, 1, 1), (2, 3, 2), (1, 2, 3), (2, 4, 6)] MST weight: 12 Non-MST edges: [(0, 2, 5), (1, 3, 4), (3, 4, 7), (0, 4, 9)] Best alternative swap: add (1, 3, 4) remove (1, 2, 3) Resulting tree weight: 13 </pre>

Prim's Algorithm — 5 Questions

Q1. Apply Prim's Algorithm starting from vertex 0. List edges added in order.

Easy Basic Prim's traversal

Input:

```
n = 4, start = 0 edges = [(0,1,10), (0,2,6), (0,3,5), (1,3,15), (2,3,4)]
```

Expected Output:

```
Step 1: Add (0,3,5) Step 2: Add (3,2,4) Step 3: Add (0,1,10) MST edges:
[(0,3,5), (3,2,4), (0,1,10)] Total weight: 19
```

main.py	Output
<pre> 1 import heapq 2 3 def prim(n,edges,start=0): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((w,v,u)); g[v].append((w,u,v)) 7 seen=[False]*n; seen[start]=True 8 pq=[] 9 for item in g[start]: heapq.heappush(pq,item) 10 mst=[] 11 while pq and len(mst)<n-1: 12 w,v,u=heapq.heappop(pq) 13 if seen[v]: continue 14 seen[v]=True; mst.append((u,v,w)) 15 for item in g[v]: 16 if not seen[item[1]]: heapq.heappush(pq,item) 17 return mst,sum(w for _,_,w in mst) 18 19 n=4; start=0 20 edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)] 21 mst,total=prim(n,edges,start) 22 for i,e in enumerate(mst,1): print(f"Step {i}: Add {e}") 23 print("MST edges:", mst) 24 print("Total weight:", total) </pre>	<pre> Step 1: Add (0, 3, 5) Step 2: Add (3, 2, 4) Step 3: Add (0, 1, 10) MST edges: [(0, 3, 5), (3, 2, 4), (0, 1, 10)] Total weight: 19 </pre>

Q2. Apply Prim's starting from vertex 2. Show the key[] array after each step.

Medium *Prim's with key tracking*

Input:

```
n = 5, start = 2 edges =
[(0,1,2), (0,3,6), (1,2,3), (1,3,8), (1,4,5), (2,4,7), (3,4,9)]
```

Expected Output:

```

Initial key[]: [inf,inf,0,inf,inf] After picking 2: key[]=[inf,3,0,inf,7]
After picking 1: key[]=[2,3,0,8,5] After picking 0: key[]=[2,3,0,6,5]
After picking 4: key[]=[2,3,0,6,5] MST edges:
[(2,1,3), (1,0,2), (1,4,5), (0,3,6)] weight: 16

```

main.py	Output
<pre> 1 n=5; start=2 2 edges=[(0,1,2),(0,3,6),(1,2,3),(1,3,8),(1,4,5),(2,4,7),(3,4,9)] 3 g=[[]*n for _ in range(n)] 4 for u,v,w in edges: g[u][v]=g[v][u]=w 5 key=[float('inf')]*n; parent=[-1]*n; used=[False]*n 6 key[start]=0 7 def show(a): return '['+'+'.join('inf' if x==float('inf') else str(x) for x in a)+']' 8 print('Initial key[]:',show(key)) 9 order=[] 10 for _ in range(n): 11 u=min((i for i in range(n) if not used[i]), key=lambda i:(key[i],i)) 12 used[u]=True; order.append(u) 13 for v in range(n): 14 if g[u][v] and not used[v] and g[u][v]<key[v]: 15 key[v]=g[u][v]; parent[v]=u 16 if u!=start: print(f'After picking {u}: key[]={show(key)}') 17 else: print(f'After picking {u}: key[]={show(key)}') 18 mst=[(parent[v],v,key[v]) for v in range(n) if v!=start] 19 print('MST edges:',mst) 20 print('weight:',sum(w for _,_,w in mst)) </pre>	<pre> Initial key[]: [inf,inf,0,inf,inf] After picking 2: key[]=[inf,3,0,inf,7] After picking 1: key[]=[2,3,0,8,5] After picking 0: key[]=[2,3,0,6,5] After picking 4: key[]=[2,3,0,6,5] After picking 3: key[]=[2,3,0,6,5] MST edges: [(1, 0, 2), (2, 1, 3), (0, 3, 6), (1, 4, 5)] weight: 16 </pre>

Q3. Apply Prim's Algorithm. Compare the result with Kruskal's on the same graph — are they identical?

Medium *Prim's vs Kruskal's comparison*

Input:

```
n = 6, start = 0 edges =
[(0,1,4), (0,2,3), (1,2,1), (1,3,2), (2,3,4), (3,4,2), (4,5,6), (3,5,5)]
```

Expected Output:

```
Prim's MST: [(0,2,3), (1,2,1), (1,3,2), (3,4,2), (3,5,5)] weight: 13
Kruskal's MST: [(1,2,1), (1,3,2), (3,4,2), (0,2,3), (3,5,5)] weight: 13 Edge
sets are identical (same total weight, same edges — different insertion
order).
```

main.py	Output
<pre> 1 import heapq 2 3 def prim(n,edges,start=0): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((w,v,u)); g[v].append((w,u,v)) 7 seen=[False]*n; seen[start]=True 8 pq=[] 9 for item in g[start]: heapq.heappush(pq,item) 10 mst=[] 11 while pq and len(mst)<n-1: 12 w,v,u=heapq.heappop(pq) 13 if seen[v]: continue 14 seen[v]=True; mst.append((u,v,w)) 15 for item in g[v]: 16 if not seen[item[1]]: heapq.heappush(pq,item) 17 return mst,sum(w for _,_,w in mst) 18 19 class DSU: 20 def __init__(self,n): 21 self.p=list(range(n)); self.r=[0]*n 22 def find(self,x): 23 while self.p[x]!=x: 24 self.p[x]=self.p[self.p[x]]; x=self.p[x] 25 return x 26 def union(self,a,b): 27 a=self.find(a); b=self.find(b) 28 if a==b: return False 29 if self.r[a]<self.r[b]: a,b=b,a 30 self.p[b]=a 31 if self.r[a]==self.r[b]: self.r[a]+=1 32 return True 33 34 def kruskal(n,edges): 35 d=DSU(n); mst=[] 36 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 37 if u!=v and d.union(u,v): mst.append((u,v,w)) 38 return mst,sum(w for _,_,w in mst) 39 40 n=6; start=0 41 edges=[(0,1,4),(0,2,3),(1,2,1),(1,3,2),(2,3,4),(3,4,2),(4,5,6),(3,5,5)] 42 pm,pw=prim(n,edges,start); km,kw=kruskal(n,edges) 43 print("Prim's MST:", pm, "weight:", pw) 44 print("Kruskal's MST:", km, "weight:", kw) 45 ps={frozenset((u,v)) (frozenset([w])) for u,v,w in []} 46 def norm(es): return {(min(u,v),max(u,v),w) for u,v,w in es} 47 print("Edge sets are identical:", norm(pm)==norm(km)) </pre>	<pre> Prim's MST: [(0, 2, 3), (2, 1, 1), (1, 3, 2), (3, 4, 2), (3, 5, 5)] weight: 13 Kruskal's MST: [(1, 2, 1), (1, 3, 2), (3, 4, 2), (0, 2, 3), (3, 5, 5)] weight: 13 Edge sets are identical: True </pre>

Q4. Run Prim's starting from each vertex (0, 1, 2). Do all starting vertices yield the same MST weight?

Medium *Starting vertex independence*

Input:

```
n = 4 edges = [(0,1,3), (0,3,7), (1,2,2), (1,3,5), (2,3,1)]
```

Expected Output:

```
MST weight (start=0): 6 edges: [(0,1,3), (1,2,2), (2,3,1)] MST weight
(start=1): 6 edges: [(1,2,2), (2,3,1), (0,1,3)] MST weight (start=2): 6
edges: [(2,3,1), (1,2,2), (0,1,3)] Yes – same total weight 6 regardless of
start vertex.
```

main.py	Output
<pre>1 import heapq 2 3 def prim(n,edges,start=0): 4 g=[[] for _ in range(n)] 5 for u,v,w in edges: 6 g[u].append((w,v,u)); g[v].append((w,u,v)) 7 seen=[False]*n; seen[start]=True 8 pq=[] 9 for item in g[start]: heapq.heappush(pq,item) 10 mst=[] 11 while pq and len(mst)<n-1: 12 w,v,u=heapq.heappop(pq) 13 if seen[v]: continue 14 seen[v]=True; mst.append((u,v,w)) 15 for item in g[v]: 16 if not seen[item[1]]: heapq.heappush(pq,item) 17 return mst,sum(w for _,_,w in mst) 18 19 n=4 20 edges=[(0,1,3),(0,3,7),(1,2,2),(1,3,5),(2,3,1)] 21 weights=[] 22 for s in [0,1,2]: 23 mst,total=prim(n,edges,s); weights.append(total) 24 print(f"MST weight (start={s}): {total} edges: {mst}") 25 print("Same total weight regardless of start:", len(set(weights))==1)</pre>	<pre>MST weight (start=0): 6 edges: [(0, 1, 3), (1, 2, 2), (2, 3, 1)] MST weight (start=1): 6 edges: [(1, 2, 2), (2, 3, 1), (1, 0, 3)] MST weight (start=2): 6 edges: [(2, 3, 1), (2, 1, 2), (1, 0, 3)] Same total weight regardless of start: True</pre>

Q5. Run Prim's on a dense graph (adjacency matrix given). Output the parent array and MST weight.

Hard *Prim's on adjacency matrix*

Input:

```
n = 5 Matrix:   [0, 2, 0, 6, 0]   [2, 0, 3, 8, 5]   [0, 3, 0, 0, 7]   [6,
8, 0, 0, 9]   [0, 5, 7, 9, 0]   (0 = no edge)
```

Expected Output:

```
parent[]: [-1, 0, 1, 0, 1] MST edges: (0,1,2), (1,2,3), (0,3,6), (1,4,5)
Total weight: 16
```

main.py	Output
<pre>1 n=5 2 g=[[0,2,0,6,0],[2,0,3,8,5],[0,3,0,0,7],[6,8,0,0,9],[0,5,7,9,0]] 3 key=[float('inf')]*n; parent=[-1]*n; used=[False]*n 4 key[0]=0 5 for _ in range(n): 6 u=min((i for i in range(n) if not used[i]), key=lambda i:key[i]) 7 used[u]=True 8 for v,w in enumerate(g[u]): 9 if w and not used[v] and w<key[v]: key[v]=w; parent[v]=u 10 edges=[(parent[v],v,g[parent[v]][v]) for v in range(1,n)] 11 print('parent[]:',parent) 12 print('MST edges:',edges) 13 print('Total weight:',sum(w for _,_,w in edges))</pre>	<pre>parent[]: [-1, 0, 1, 0, 1] MST edges: [(0, 1, 2), (1, 2, 3), (0, 3, 6), (1, 4, 5)] Total weight: 16</pre>

Boruvka's Algorithm — 5 Questions

Q1. Apply Boruvka's Algorithm. Show the cheapest edge selected for each component in the first pass.

Easy *Basic Boruvka's pass*

Input:

`n = 4, m = 5 edges = [(0,1,10), (0,2,6), (0,3,5), (1,3,15), (2,3,4)]`

Expected Output:

Pass 1 cheapest edges: Component {0}: cheapest = (0,3,5) Component {2}: cheapest = (2,3,4) Component {3}: cheapest = (2,3,4) After pass 1 merges: {0,2,3} merged, {1} separate Pass 2: add (0,1,10) MST edges: [(2,3,4), (0,3,5), (0,1,10)] weight: 19

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): self.p=list(range(n)); self.sz=[1]*n 3 def find(self,x): 4 if self.p[x]!=x: self.p[x]=self.find(self.p[x]) 5 return self.p[x] 6 def union(self,a,b): 7 a=self.find(a); b=self.find(b) 8 if a==b: return False 9 if self.sz[a]<self.sz[b]: a,b=b,a 10 self.p[b]=a; self.sz[a]+=self.sz[b]; return True 11 12 def boruvka(n,edges,trace=False): 13 d=DSU(n); mst=[]; phases=[] 14 while len(mst)<n-1: 15 cheapest={} 16 for e in edges: 17 u,v,w=e; a=d.find(u); b=d.find(v) 18 if a==b: continue 19 for c in (a,b): 20 if c not in cheapest or (w,u,v)<(cheapest[c][2],cheapest[c][0],cheapest[c][1]): cheapest[c]=e 21 if not cheapest: break 22 added=[] 23 for e in sorted(set(cheapest.values()), key=lambda x:(x[2],x[0],x[1])): 24 if d.union(e[0],e[1]): mst.append(e); added.append(e) 25 phases.append((cheapest.copy(),added.copy())) 26 if not added: break 27 return mst,sum(w for _,_,w in mst),phases,d 28 29 n=4 30 edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)] 31 mst,total,phases,d=boruvka(n,edges) 32 ch,added=phases[0] 33 print('Pass 1 cheapest edges:') 34 for c in sorted(ch): print(f'Component {{c}}: cheapest = {ch[c]}') 35 print('Edges added in pass 1:',added) 36 if len(phases)>1: print('Pass 2 edges added:',phases[1][1]) 37 print('MST edges:',mst) 38 print('weight:',total) </pre>	<pre> Pass 1 cheapest edges: Component {0}: cheapest = (0, 3, 5) Component {1}: cheapest = (0, 1, 10) Component {2}: cheapest = (2, 3, 4) Component {3}: cheapest = (2, 3, 4) Edges added in pass 1: [(2, 3, 4), (0, 3, 5), (0, 1, 10)] MST edges: [(2, 3, 4), (0, 3, 5), (0, 1, 10)] weight: 19 </pre>

Q2. Run Boruvka's Algorithm. State how many passes (phases) are needed and which edges are added each phase.

Easy Phase-by-phase trace

Input:

```
n = 5, m = 7 edges =  
[(0,1,2), (0,3,6), (1,2,3), (1,3,8), (1,4,5), (2,4,7), (3,4,9)]
```

Expected Output:

```
Phase 1 edges added: [(0,1,2), (1,2,3), (1,4,5), (0,3,6)] All vertices merged  
-> done in 1 phase! MST: [(0,1,2), (1,2,3), (1,4,5), (0,3,6)] weight: 16
```

main.py	Output
<pre>1 class DSU: 2 def __init__(self,n): self.p=list(range(n)); self.sz=[1]*n 3 def find(self,x): 4 if self.p[x]!=x: self.p[x]=self.find(self.p[x]) 5 return self.p[x] 6 def union(self,a,b): 7 a=self.find(a); b=self.find(b) 8 if a==b: return False 9 if self.sz[a]<self.sz[b]: a,b=b,a 10 self.p[b]=a; self.sz[a]+=self.sz[b]; return True 11 12 def boruvka(n,edges,trace=False): 13 d=DSU(n); mst=[]; phases=[] 14 while len(mst)<n-1: 15 cheapest={} 16 for e in edges: 17 u,v,w=e; a=d.find(u); b=d.find(v) 18 if a==b: continue 19 for c in (a,b): 20 if c not in cheapest or (w,u,v)<(cheapest[c][2],cheapest[c][0],cheapest[c][1]): cheapest[c]=e 21 if not cheapest: break 22 added=[] 23 for e in sorted(set(cheapest.values()), key=lambda x:(x[2],x[0],x[1])): 24 if d.union(e[0],e[1]): mst.append(e); added.append(e) 25 phases.append((cheapest.copy(),added.copy())) 26 if not added: break 27 return mst,sum(w for _,_,w in mst),phases,d 28 29 n=5 30 edges=[(0,1,2),(0,3,6),(1,2,3),(1,3,8),(1,4,5),(2,4,7),(3,4,9)] 31 mst,total,phases,d=boruvka(n,edges) 32 for i,(_,added) in enumerate(phases,1): print(f'Phase {i} edges added:',added) 33 print('Phases needed:',len(phases)) 34 print('MST:',mst) 35 print('weight:',total)</pre>	<pre>Phase 1 edges added: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)] Phases needed: 1 MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)] weight: 16</pre>

Q3. Apply Boruvka's Algorithm and trace the Union-Find structure after each phase.

Medium Boruvka's with Union-Find trace

Input:

```
n = 6, m = 9 edges =  
[(0,1,4), (0,2,3), (1,2,1), (1,3,2), (2,3,4), (3,4,2), (4,5,6), (3,5,5), (0,5,9)]
```

Expected Output:

```
Phase 1 cheapest per component: {0}→(0,2,3) {1}→(1,2,1) {2}→(1,2,1)
{3}→(1,3,2) {4}→(3,4,2) {5}→(4,5,6) Edges added:
(1,2,1),(1,3,2),(3,4,2),(0,2,3),(4,5,6) After phase 1: one component
{0,1,2,3,4,5} MST weight: 14
```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): self.p=list(range(n)); self.sz=[1]*n 3 def find(self,x): 4 if self.p[x]!=x: self.p[x]=self.find(self.p[x]) 5 return self.p[x] 6 def union(self,a,b): 7 a=self.find(a); b=self.find(b) 8 if a==b: return False 9 if self.sz[a]<self.sz[b]: a,b=b,a 10 self.p[b]=a; self.sz[a]+=self.sz[b]; return True 11 12 def boruvka(n,edges,trace=False): 13 d=DSU(n); mst=[]; phases=[] 14 while len(mst)<n-1: 15 cheapest={} 16 for e in edges: 17 u,v,w=e; a=d.find(u); b=d.find(v) 18 if a==b: continue 19 for c in (a,b): 20 if c not in cheapest or (w,u,v)<(cheapest[c][2],cheapest[c][0],cheapest[c][1]): cheapest[c]=e 21 if not cheapest: break 22 added=[] 23 for e in sorted(set(cheapest.values()), key=lambda x:(x[2],x[0],x[1])): 24 if d.union(e[0],e[1]): mst.append(e); added.append(e) 25 phases.append((cheapest.copy(),added.copy())) 26 if not added: break 27 return mst,sum(w for _,_,w in mst),phases,d 28 29 n=6 30 edges=[(0,1,4),(0,2,3),(1,2,1),(1,3,2),(2,3,4),(3,4,2),(4,5,6),(3,5,5),(0,5,9)] 31 mst,total,phases,d=boruvka(n,edges) 32 for i,(ch,added) in enumerate(phases,1): 33 print(f'Phase {i} cheapest per component:') 34 for c in sorted(ch): print(f' {{c}} -> {ch[c]}') 35 print('Edges added:',added) 36 print('MST edges:',mst) 37 print('MST weight:',total) </pre>	<pre> Phase 1 cheapest per component: {0} -> (0, 2, 3) {1} -> (1, 2, 1) {2} -> (1, 2, 1) {3} -> (1, 3, 2) {4} -> (3, 4, 2) {5} -> (3, 5, 5) Edges added: [(1, 2, 1), (1, 3, 2), (3, 4, 2), (0, 2, 3), (3, 5, 5)] MST edges: [(1, 2, 1), (1, 3, 2), (3, 4, 2), (0, 2, 3), (3, 5, 5)] MST weight: 13 </pre>

Q4. Two edges share the same minimum weight. Explain how Boruvka's handles ties and find the MST.

Medium Tie-breaking in Boruvka's

Input:

```
n = 4, m = 6 edges = [(0,1,5),(0,2,5),(1,2,3),(1,3,4),(2,3,4),(0,3,8)]
```

Expected Output:

```

Phase 1:  {0}: cheapest = (0,1,5) [first encountered on tie]  {1}:
cheapest = (1,2,3)  {2}: cheapest = (1,2,3)  {3}: cheapest = (1,3,4)
[first encountered on tie] Edges added: (1,2,3),(1,3,4),(0,1,5) Phase 2:
all connected MST: [(1,2,3),(1,3,4),(0,1,5)] weight: 12

```

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): self.p=list(range(n)); self.sz=[1]*n 3 def find(self,x): 4 if self.p[x]!=x: self.p[x]=self.find(self.p[x]) 5 return self.p[x] 6 def union(self,a,b): 7 a=self.find(a); b=self.find(b) 8 if a==b: return False 9 if self.sz[a]<self.sz[b]: a,b=b,a 10 self.p[b]=a; self.sz[a]+=self.sz[b]; return True 11 12 def boruvka(n,edges,trace=False): 13 d=DSU(n); mst=[]; phases=[] 14 while len(mst)<n-1: 15 cheapest={} 16 for e in edges: 17 u,v,w=e; a=d.find(u); b=d.find(v) 18 if a==b: continue 19 for c in (a,b): 20 if c not in cheapest or (w,u,v)<(cheapest[c][2],cheapest[c][0],cheapest[c][1]): cheapest[c]=e 21 if not cheapest: break 22 added=[] 23 for e in sorted(set(cheapest.values()), key=lambda x:(x[2],x[0],x[1])): 24 if d.union(e[0],e[1]): mst.append(e); added.append(e) 25 phases.append((cheapest.copy(),added.copy())) 26 if not added: break 27 return mst,sum(w for _,_,w in mst),phases,d 28 29 n=4 30 edges=[(0,1,5),(0,2,5),(1,2,3),(1,3,4),(2,3,4),(0,3,8)] 31 # Preserve input order on equal weights for each component. 32 def boruvka_first_tie(n,edges): 33 d=DSU(n); mst=[]; phases=[] 34 while len(mst)<n-1: 35 cheapest={} 36 for e in edges: 37 u,v,w=e; a=d.find(u); b=d.find(v) 38 if a==b: continue 39 for c in (a,b): 40 if c not in cheapest or w<cheapest[c][2]: cheapest[c]=e 41 added=[] 42 for e in dict.fromkeys(cheapest.values()): 43 if d.union(e[0],e[1]): mst.append(e); added.append(e) 44 phases.append((cheapest,added)) 45 if not added: break 46 return mst,sum(e[2] for e in mst),phases 47 mst,total,phases=boruvka_first_tie(n,edges) 48 for i,(ch,added) in enumerate(phases,1): 49 print(f'Phase {i}:') 50 for c in sorted(ch): print(f' Component {{{c}}}: cheapest = {ch[c]}') 51 print(' Edges added:',added) 52 print('MST:',mst) 53 print('weight:',total) </pre>	<pre> Phase 1: Component {0}: cheapest = (0, 1, 5) Component {1}: cheapest = (1, 2, 3) Component {2}: cheapest = (1, 2, 3) Component {3}: cheapest = (1, 3, 4) Edges added: [(0, 1, 5), (1, 2, 3), (1, 3, 4)] MST: [(0, 1, 5), (1, 2, 3), (1, 3, 4)] weight: 12 </pre>

Q5. Compare Boruvka's, Kruskal's and Prim's on this graph. Which runs the fewest phases / iterations?

Hard Algorithm comparison

Input:

```
n = 6, m = 10 edges = [(0,1,6), (0,2,1), (0,3,5), (1,2,5), (1,4,3), (2,3,5),
(2,4,6), (2,5,4), (3,5,2), (4,5,6)]
```

Expected Output:

```
MST edges (all 3 algorithms agree):
[(0,2,1), (3,5,2), (1,4,3), (2,5,4), (0,3,5)] weight: 15 Boruvka's: 2
phases Kruskal's: 5 edge selections (sorted pass) Prim's: 5 vertex
```

additions Boruvka's fewest 'rounds' due to parallel component merging.
For dense graphs ($m \gg n$), Prim's with a binary heap is typically fastest.

main.py	Output
<pre> 1 class DSU: 2 def __init__(self,n): self.p=list(range(n)); self.sz=[1]*n 3 def find(self,x): 4 if self.p[x]!=x: self.p[x]=self.find(self.p[x]) 5 return self.p[x] 6 def union(self,a,b): 7 a=self.find(a); b=self.find(b) 8 if a==b: return False 9 if self.sz[a]<self.sz[b]: a,b=b,a 10 self.p[b]=a; self.sz[a]+=self.sz[b]; return True 11 12 def boruvka(n,edges,trace=False): 13 d=DSU(n); mst=[]; phases=[] 14 while len(mst)<n-1: 15 cheapest={} 16 for e in edges: 17 u,v,w=e; a=d.find(u); b=d.find(v) 18 if a==b: continue 19 for c in (a,b): 20 if c not in cheapest or (w,u,v)<(cheapest[c][2],cheapest[c][0],cheapest[c][1]): cheapest[c]=e 21 if not cheapest: break 22 added=[] 23 for e in sorted(set(cheapest.values()), key=lambda x:(x[2],x[0],x[1])): 24 if d.union(e[0],e[1]): mst.append(e); added.append(e) 25 phases.append((cheapest.copy(),added.copy())) 26 if not added: break 27 return mst,sum(w for _,_,w in mst),phases,d 28 29 class DSU: 30 def __init__(self,n): 31 self.p=list(range(n)); self.r=[0]*n 32 def find(self,x): 33 while self.p[x]!=x: 34 self.p[x]=self.p[self.p[x]]; x=self.p[x] 35 return x 36 def union(self,a,b): 37 a=self.find(a); b=self.find(b) 38 if a==b: return False 39 if self.r[a]<self.r[b]: a,b=b,a 40 self.p[b]=a 41 if self.r[a]==self.r[b]: self.r[a]+=1 42 return True 43 44 def kruskal(n,edges): 45 d=DSU(n); mst=[] 46 for u,v,w in sorted(edges,key=lambda e:(e[2],e[0],e[1])): 47 if u!=v and d.union(u,v): mst.append((u,v,w)) 48 return mst,sum(w for _,_,w in mst) 49 50 import heapq 51 52 def prim(n,edges,start=0): 53 g=[[] for _ in range(n)] 54 for u,v,w in edges: 55 g[u].append((w,v,u)); g[v].append((w,u,v)) 56 seen=[False]*n; seen[start]=True 57 pq=[] 58 for item in g[start]: heapq.heappush(pq,item) 59 mst=[] 60 while pq and len(mst)<n-1: 61 w,v,u=heapq.heappop(pq) 62 if seen[v]: continue 63 seen[v]=True; mst.append((u,v,w)) 64 for item in g[v]: 65 if not seen[item[1]]: heapq.heappush(pq,item) 66 return mst,sum(w for _,_,w in mst) 67 68 n=6 69 edges=[(0,1,6),(0,2,1),(0,3,5),(1,2,5),(1,4,3),(2,3,5),(2,4,6),(2,5,4),(3,5,2),(4,5,6)] 70 bm,bw,phases,_,_=boruvka(n,edges) 71 km,kw=kruskal(n,edges) 72 pm,pw=prim(n,edges,0) 73 print("Boruvka MST:",bm,"weight:",bw) 74 print("Kruskal MST:",km,"weight:",kw) 75 print("Prim MST:",pm,"weight:",pw) 76 print("Boruvka phases:",len(phases)) 77 print("Kruskal edge selections:",len(km)) 78 print("Prim vertex additions:",len(pm)) 79 print("Fewest rounds: Boruvka") </pre>	<pre> Boruvka MST: [(0, 2, 1), (3, 5, 2), (1, 4, 3), (2, 5, 4), (1, 2, 5)] weight: 15 Kruskal MST: [(0, 2, 1), (3, 5, 2), (1, 4, 3), (2, 5, 4), (1, 2, 5)] weight: 15 Prim MST: [(0, 2, 1), (2, 5, 4), (5, 3, 2), (2, 1, 5), (1, 4, 3)] weight: 15 Boruvka phases: 2 Kruskal edge selections: 5 Prim vertex additions: 5 Fewest rounds: Boruvka </pre>